

Ahead of Time Optimizations

Ahead of Time Optimizations

Best Practices

- Annotate your Domain Types

- Ahead of Time Code Generation

- Ahead of Time Repositories

- Repository Metadata

- Native Image Runtime Hints

- JPA Ahead of Time Repositories

This chapter covers Spring Data's Ahead of Time (AOT) optimizations that build upon [Spring's Ahead of Time Optimizations](#).

Best Practices

Annotate your Domain Types

During application startup, Spring scans the classpath for domain classes for early processing of entities. By annotating your domain types with Spring Data Store specific `@Table`, `@Document` or `@Entity` annotations you can aid initial entity scanning and ensure that those types are registered with `ManagedTypes` for Runtime Hints. Classpath scanning is not possible in native image arrangements and so Spring has to use `ManagedTypes` for the initial entity set.

Ahead of Time Code Generation

Ahead of time code generation is not limited to usage with GraalVM Native Image but also offers benefits when working with regular deployments and can help optimize startup performance on the jvm.

NOTE

With AOT optimizations some decisions (like database dialects for example) will be frozen at build time and get included as is in the application setup.

If Ahead of Time compilation is enabled Spring Data can (depending on the actual Module in use) contribute several components during the AOT phase of your build.

- Bytecode for generated Type/Property Accessors
- Sourcecode for the defined Repository Interfaces
- Repository Metadata in JSON format

Each of the above is enabled by default. However, there users may fine tune the configuration with following options.

<code>spring.aot.data.accessors.enabled</code>	Boolean flag to control contribution of Bytecode for generated Type/Property Accessors
<code>spring.aot.data.accessors.include</code>	Comma separated list of FQCN for which to contribute Bytecode for generated Type/Property Accessors. Ant-style include patterns matching package names (e.g. <code>example.springdata.**</code>) or type names inclusion. If a type is matched by an inclusion and an exclusion, the inclusion wins and the type is considered included.
<code>spring.aot.data.accessors.exclude</code>	Comma separated list of FQCN for which to skip contribution of Bytecode for generated Type/Property Accessors. Ant-style exclude patterns matching package names (e.g. <code>example.springdata.**</code>) or type names exclusion. If a type is matched by an inclusion and an exclusion, the inclusion wins and the type is considered included.
<code>spring.aot.repositories.enabled</code>	Boolean flag to control contribution of Source Code for Repository Interfaces
<code>spring.aot.[module-name].repositories.enabled</code>	Boolean flag to control contribution of Source Code for Repository Interfaces for a certain module (e.g. <code>cassandra</code> , <code>jdbc</code> , <code>jpa</code> , <code>mongodb</code>)
<code>spring.aot.repositories.metadata.enabled</code>	Boolean flag to control contribution of JSON repository metadata containing query methods and actual query strings. Requires <code>spring.aot.repositories.enabled</code> to be enabled.

Ahead of Time Repositories

NOTE

Ahead of Time repositories are only available for imperative (non reactive) repository interfaces of certain modules. The criteria identifying eligible query methods varies between the implementing modules.

AOT Repositories are an extension to AOT processing by pre-generating eligible query method implementations. Query methods are opaque to developers regarding their underlying queries being exe-

cuted in a query method call. AOT repositories contribute query method implementations based on derived, annotated, and named queries that are known at build-time. This optimization moves query method processing from runtime to build-time, which can lead to a significant performance improvement as query methods do not need to be analyzed reflectively upon each application start.

The resulting AOT repository fragment follows the naming scheme of `<Repository FQCN>Impl__AotRepository` and is placed in the same package as the repository interface.

WARNING

Consider AOT repository classes an internal optimization. Do not use them directly in your code as generation and implementation details may change in future releases.

Repository Metadata

AOT processing introspects query methods and collects metadata about repository queries. Spring Data stores this metadata in JSON files that are named after the source repository within the same package. Repository JSON Metadata contains details about queries and fragments. An example for the following repository is shown below:

Metadata	Repository
<pre>{ "name": "example.springdata.UserRepository", "module": "JDBC", "type": "IMPERATIVE", "methods": [{ "name": "findBy", "signature": "public abstract java.util.List<example.springdata.User> example.springdata.UserRepository.findBy()", "query": { "query": "SELECT * FROM User" } }, { "name": "findByLastnameStartingWith", "signature": "public abstract org.springframework.data.domain.Page<example.springdata.User> example.springdata.UserRepository.findByLastnameStartingWith(java.lang.String,org.springframework.data.domain.Pageable)", "query": { "query": "SELECT * FROM User u WHERE lastname LIKE :lastname", "count-query": "SELECT COUNT(*) FROM User WHERE lastname LIKE :lastname" } }] }</pre>	

```
{
  "name": "findByEmailAddress",
  "signature": "public abstract example.springdata.User
example.springdata.UserRepository.findByEmailAddress(java.lang.String)",
  "query": {
    "query": "select * from User where emailAddress = ?1"
  }
},
```

| NOTE

Creating JSON metadata can be controlled via the `spring.aot.repositories.metadata.enabled` flag.

Native Image Runtime Hints

Running an application as a native image requires additional information compared to a regular JVM runtime. Spring Data contributes [Runtime Hints](#) during AOT processing for native image usage. These are in particular hints for:

- Auditing
- `ManagedTypes` to capture the outcome of class-path scans
- Repositories
 - Reflection hints for entities, return types, and Spring Data annotations
 - Repository fragments
 - Querydsl Q classes
 - Kotlin Coroutine support
- Web support (Jackson Hints for `PagedModel`)

JPA Ahead of Time Repositories

AOT repositories filter methods that are eligible for AOT processing. These are typically all query methods that are not backed by an [implementation fragment](#).

| NOTE

AOT processing avoids database access. Therefore, it initializes an in-memory Hibernate instance for metadata collection. Types for the Hibernate configuration are determined by our AOT metadata collector. We prefer using a `PersistentEntityTypes` bean if available and fall back to `PersistenceUnitInfo` or our own discovered types. If our type scanning is not sufficient for your arrangement, you can enable direct `EntityManagerFactory` usage by configuring the `spring.aot.jpa.repositories.use-entitymanager=true` property.

Supported Features

- Derived query methods, `@Query` / `@NativeQuery` and named query methods
- Stored procedure query methods annotated with `@Procedure`
- `@Modifying` methods returning `void` or `int`
- `@QueryHints` support
- `Pagination`, `Slice`, `Stream`, and `Optional` return types
- Sort query rewriting
- Interface and DTO Projections
- Value Expressions (Those require a bit of reflective information. Mind that using Value Expressions requires expression parsing and contextual information to evaluate the expression)

Limitations

- Requires Hibernate for AOT processing.
- `QueryRewriter` must be a no-args class. `QueryRewriter` beans are not yet supported.
- Methods accepting `ScrollPosition` (e.g. `Keyset` pagination) are not yet supported.
- Custom Collection return types (e.g. `io.vavr.collection`, `"org.eclipse.collections"`) are not yet supported.

Excluded methods

- `CrudRepository`, `Querydsl`, `Query by Example`, and other base interface methods as their implementation is provided by the base class respective fragments
- Methods whose implementation would be overly complex
 - Methods accepting `ScrollPosition` (e.g. `Keyset` pagination)
 - Dynamic projections



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle

and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. “AWS” and “Amazon Web Services” are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.